

Choose the Realtime model you are building with. For GPT-Live, use [Prompting GPT-Live](#).

`gpt-realtime-2`

`gpt-realtime-1.5`

`gpt-realtime-2` is our state-of-the-art reasoning voice model for low-latency speech-to-speech applications. It can think before it speaks, follow instructions more reliably, use a larger context window, and call tools with greater precision than earlier realtime models.

To take advantage of these gains, design prompts with more intent. Explicitly define the assistant’s responsibilities, decision points, tool-calling behavior, and guardrails: what it should do, when it should do it, and what it should avoid.

 Start simple. Do not over-prompt upfront. Begin with a minimal prompt, run evaluations, then add instructions only for behaviors that fail in testing.

Choose a model

Model	Use when	Prompting focus
<code>gpt-realtime-2</code>	You need the strongest realtime reasoning, tool use, and instruction following.	Tune reasoning effort, preambles, tool policies, exact entity capture, and long-session state.
<code>gpt-realtime-1.5</code>	You need a fast, reliable non-reasoning speech-to-speech model.	Follow the core realtime prompt structure and test for latency-sensitive behavior.

Realtime 2.0 Prompting Guide

Use `gpt-realtime-2` when the voice agent needs stronger reasoning, tool selection, exact entity handling, or long-session state. Start with `reasoning.effort: "low"`, test default preamble behavior, and define clear confirmation boundaries before write actions.

Loading docs agent...

Recommended prompt structure

Use short, labeled sections. The model should be able to find the relevant instructions quickly.

Role and Objective



Personality and Tone

Language

Reasoning

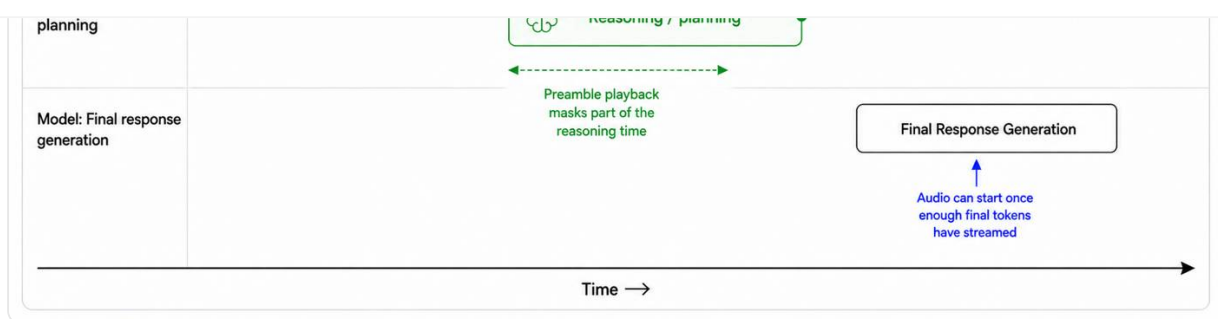
Message Channels

Loading docs agent...

low	You need responsiveness plus basic reasoning.	Customer support, order lookup, simple policy questions.
medium	The assistant must reason through multi-step tasks.	Technical support, diagnostics, complex routing.
high	Deeper reasoning materially improves success.	High-precision workflows, escalation decisions, tasks with constraints.
xhigh	Maximum reasoning is worth added latency and cost.	Complex planning, critical triage, high-stakes tool orchestration.

Beyond the API setting, steer the model on when and how much to reason.

Loading docs agent...



Preambles



Use short preambles only when they help the user understand that work is happening.

When to use a preamble

Use a preamble when:

Loading docs agent...

Avoid phrases like:

- "Let me think..."
- "Hmm..."
- "One moment while I process that..."
- "I am now going to access the tool..."

Preamble length

Use one short sentence.

Do not exceed two short sentences unless the user needs an explanation before a high-impact action.

Prefer

- "I'll check that order now."

Loading docs agent...

for the full procedure.

- Escalations: Briefly explain why escalation is needed and what will happen next.

Example:

“User: Which plan should I choose?”

“Assistant: If you want the lowest cost, choose Basic. If you need team permissions and shared billing, choose Pro. If compliance review or admin controls matter, choose Enterprise.”

Design tool behavior

Loading docs agent...

TOOLS

LP

Use only the tools explicitly provided in the current tool list.
Do not invent, assume, simulate, or rename tools.

For read-only tools:

- Call the tool when the user's intent is clear and all required fields are available.
- Do not ask for confirmation unless the lookup depends on a high-precision identifier or there is meaningful risk of using the wrong record.
- Ask a clarification question only if a required field is missing, ambiguous, or conflicting.

For write tools or external actions:

- Summarize the intended action before calling the tool.

Loading docs agent...

“Assistant: To confirm, you want me to charge the card on file \$248.16 for the remaining balance. Should I proceed?”

Recover from tool failures

Tool failures are part of the conversation. A good recovery should explain what happened and give the user a clear next step.

Do not treat every failure the same. Recovery behavior should depend on the tool type, failure mode, and user impact. Some failures should be handled silently with a retry. Others require asking the user to clarify, correct an identifier, confirm a new action, or choose an alternate path.

Loading docs agent...

Realtime models are eager to help. If the prompt mentions a tool that is not actually available, or if the tool list does not match the prompt, the model may invent a tool name or pretend it completed the action.

For example, if the prompt references `lookup_order`, but the provided tool is named `search_orders`, the model may call the wrong name or simulate the action.

Tool Availability



Use only the tools that are explicitly provided in the current tool list.

Do not invent, assume, or simulate tools. If a tool is mentioned in the instructions but is not present in the tool list, treat it as unavailable.

Loading docs agent...

```
3     "description": "Call this when the latest audio does not  
4     need a spoken response, such as silence, background noise,  
5     hold music, TV audio, side conversation, or speech not  
6     addressed to the assistant. This tool helps end the turn  
7     without a spoken reply.",  
8     "parameters": {  
9         "type": "object",  
10        "properties": {},  
11        "required": []  
12    }  
13 }
```

Pair it with prompt instructions:

Handling Silence and Background Noise



If the latest audio is silence, background noise, hold music, TV

Loading docs agent...

the commentary channel.

Before calling tools in the commentary channel, briefly tell the user what you are doing. 

`gpt-realtime-2` can emit multiple response phases in a single turn. In API output, this distinction is represented by the `response.done` event, which includes a `phase` value that indicates whether the content is commentary or the final answer.

You can use this field to handle each phase differently in your application. For example, commentary can be played or displayed as a short intermediate update, while `final_answer` can be reserved for the assistant's completed response.

Loading docs agent...

channel when the audio is unclear.

Example:

“User audio: “Check order three one-” [cut off]”

Bad:

“Assistant: I’ll check order 31 now.”

Good:

“Assistant: I heard only part of the order number. Could you repeat it digit by digit?”

Loading docs agent...

Example:

"I see tracking number ABC-54321 from earlier. Should I use that one, or do you have a different tracking number?"

Do not call tools until the current value has been collected, validated, and confirmed.

Handle spelled-out characters

Use this when users spell IDs, codes, names, or email addresses one character at a time. The spoken form is input, not the final value.

Spelled-Out Characters



Loading docs agent...

- "one nineteen" -> "119"
- "ninety nine eleven" -> "9911"
- "nine thousand nine hundred eleven" -> "9911"

If multiple interpretations are plausible, ask the user to clarify before using the value.

Example:

"I heard either 119 or 1-19. Could you repeat the number digit by digit?"

Confirm exact identifiers before tool calls

Order IDs, tracking numbers, account numbers, claim numbers, confirmation codes, and similar identifiers are high-precision fields. Confirm them before using them in a

Loading docs agent...

Email addresses are important values. Dots, dashes, underscores, repeated letters, and similar-sounding names can cause account lookup failures or send messages to the wrong address.

Ask the user to spell the email address:

“Assistant: Could you spell the email address character by character so I can make sure I have it exactly right?”

When reading it back, confirm the exact final address:

“Assistant: Just to confirm, that is c-h-e-n at example dot com, right?”

Loading docs agent...

Use precise language. The model may prioritize the exact wording of an instruction over the broader behavior you intended. Broad or rigid rules can dominate the assistant's behavior in surprising ways, especially when multiple rules overlap.

Be careful with constraint words such as `must` , `only` , `never` , and `always` . Use them when the behavior is truly required, not as general emphasis. Overusing hard constraints can make the assistant rigid, overly cautious, or unable to handle reasonable exceptions.

Prefer precise scope:

For write actions that modify user data, ask for confirmation before calling the tool.



Avoid broad scope:

Loading docs agent...

with a Hindi, Spanish, French, or Mandarin accent and still expect English responses.

Avoid broad language instructions such as:

Mirror the user.
Respond naturally in the user's language.
Switch languages when appropriate.
Sound local.
Adapt to the user's accent.



These are too broad. The model may interpret accent, filler words, backchannels, or isolated foreign words as a reason to switch languages.

English language policy

Loading docs agent...

Do not switch languages based on:

- accent;
- pronunciation;
- filler words;
- short backchannels;
- names;
- addresses;
- isolated foreign words.

If uncertain, ask:

"Would you like me to continue in English or [LANGUAGE]?"

Accent control

Loading docs agent...

Custom voices

Use Custom Voices when standard voices cannot reliably meet brand, accent, or character requirements.

Prompting can steer accent, pacing, and delivery, but it cannot fully replace voice design. For use cases that require consistent branded voice identity or accent fidelity, consider Custom Voices.

Custom Voices are available only to approved customers. Contact your account team for access.

Maintain state in long sessions

`gpt-realtime-2` expands the realtime context window from 32k to 128k tokens, making it better suited for long sessions. For dense two-way conversations, 128k tokens is best thought of as roughly 1-2 hours of dense raw audio context. This will

Loading docs agent...

4. Remove stale examples. Add short examples for happy paths, ambiguity, interruptions, tool calls, and fallback behavior.
5. Compare representative conversations before and after migration. Check for regressions against an existing eval and document intentional behavior changes.
6. Run a final consistency pass. Confirm the prompt clearly separates hard requirements, defaults, tool rules, safety rules, and fallback behavior.
7. Run evals, inspect representative failures, and iterate on the prompt until the target behaviors are reliable.

Next steps

For GPT-Live:

- Review [delegation and tools](#) for the backend prompt and application-owned context.

Loading docs agent...